

CAHIER DES CHARGES DU PROJET

“V.ROOM”

I. Présentation de l'Ensemble du Projet

- **Contexte** : Dans le cadre de mon BTS, j'ai choisi de concevoir et développer une application web e-commerce complète nommée "V.ROOM". Ce projet a pour vocation de démontrer l'acquisition et la maîtrise d'une stack technique moderne (React, Symfony) et d'intégrer des technologies innovantes comme l'orchestration d'une Intelligence Artificielle.
- **Objectifs** : L'objectif principal est de développer une plateforme de vente de véhicules (neufs et d'occasion). La valeur ajoutée majeure du projet réside dans l'intégration d'un assistant IA. Plutôt que d'utiliser des filtres classiques complexes, l'utilisateur pourra exprimer son besoin en langage naturel, et l'IA se chargera de trouver le véhicule correspondant dans la base de données.
- **Cible utilisateurs** : La plateforme s'adresse à des particuliers, qu'ils soient passionnés d'automobile ou de simples curieux cherchant à acquérir un véhicule. L'interface est pensée pour des utilisateurs à l'aise avec le web mais qui ne connaissent pas forcément les critères techniques d'une voiture, justifiant ainsi le besoin d'un assistant IA pour les guider de manière vulgarisée et personnalisée.

II. Analyse des Besoins

1. Acteurs du système

- **Visiteur** : Utilisateur non connecté naviguant sur le site.
- **Client** : Utilisateur authentifié disposant d'un compte.
- **Administrateur** : Gérant de la boutique (ajoute/modifie les véhicules en base de données).

2. Besoins fonctionnels

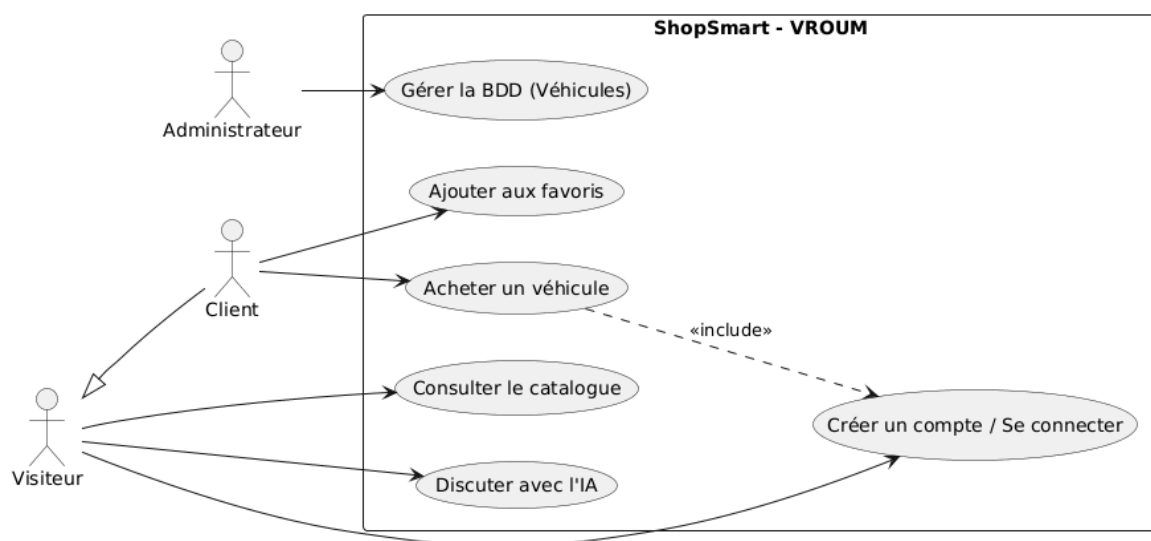
- **Gestion des utilisateurs (Authentification)** :
 - Création d'un compte client, connexion et déconnexion sécurisées.
 - Gestion du profil utilisateur.
- **Consultation du catalogue** :
 - Affichage d'une liste dynamique des véhicules disponibles.
 - Consultation de la fiche détaillée d'un véhicule.
- **Assistance par Intelligence Artificielle** :
 - Interface de chat permettant à l'utilisateur de décrire son besoin en langage naturel.
 - Analyse de la requête et recommandation de véhicules pertinents issus de la base de données.
- **Gestion des favoris** :
 - Possibilité pour un client connecté d'ajouter des véhicules à une liste de favoris pour les retrouver plus tard.
- **Processus d'achat** :
 - Ajout d'un véhicule au panier.
 - Validation du panier et simulation du processus de commande.

3. Besoins non-fonctionnels (Qualité et contraintes du système)

- **Ergonomie et Accessibilité** : Interface fluide et moderne grâce à React. Le site doit être totalement *responsive*.
- **Performances** : Temps de réponse rapide de l'API Symfony, notamment lors du filtrage des véhicules.
- **Sécurité** : Protection des mots de passe en base de données, sécurisation des routes API (seuls les utilisateurs connectés peuvent accéder à leurs favoris), et sécurisation de la clé d'API de l'Intelligence Artificielle.

4. Répartition des droits et Cas d'Utilisation

- **Visiteur** : Il a un accès en lecture au catalogue de véhicules et peut interagir avec l'assistant IA pour affiner ses recherches. Son objectif principal est la découverte. S'il souhaite aller plus loin, il doit s'inscrire.
- **Client** : Il hérite de tous les droits du visiteur. En tant qu'utilisateur authentifié, il débloquent des fonctionnalités d'écriture et de transaction : il peut sauvegarder des véhicules dans ses favoris et procéder à l'achat via le tunnel de commande.
- **Administrateur** : Il possède des droits de gestion sur le système. Il est le seul habilité à créer, modifier ou supprimer des fiches de véhicules dans la base de données.



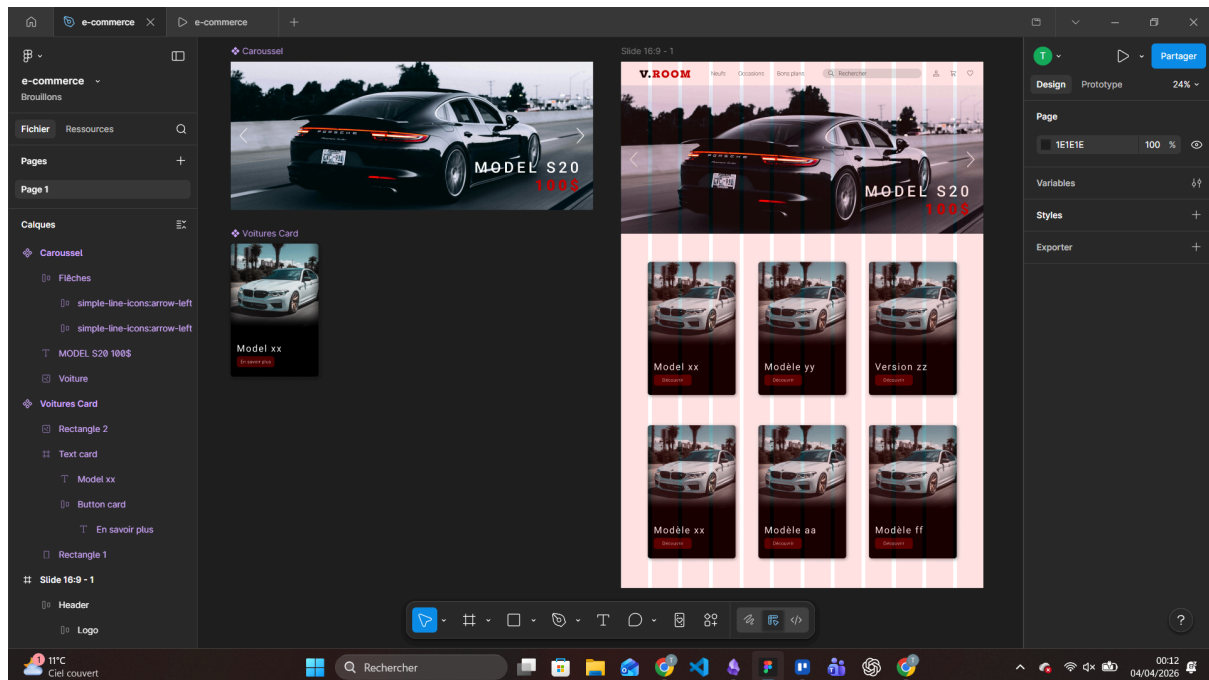
III. Description Graphique et Ergonomique

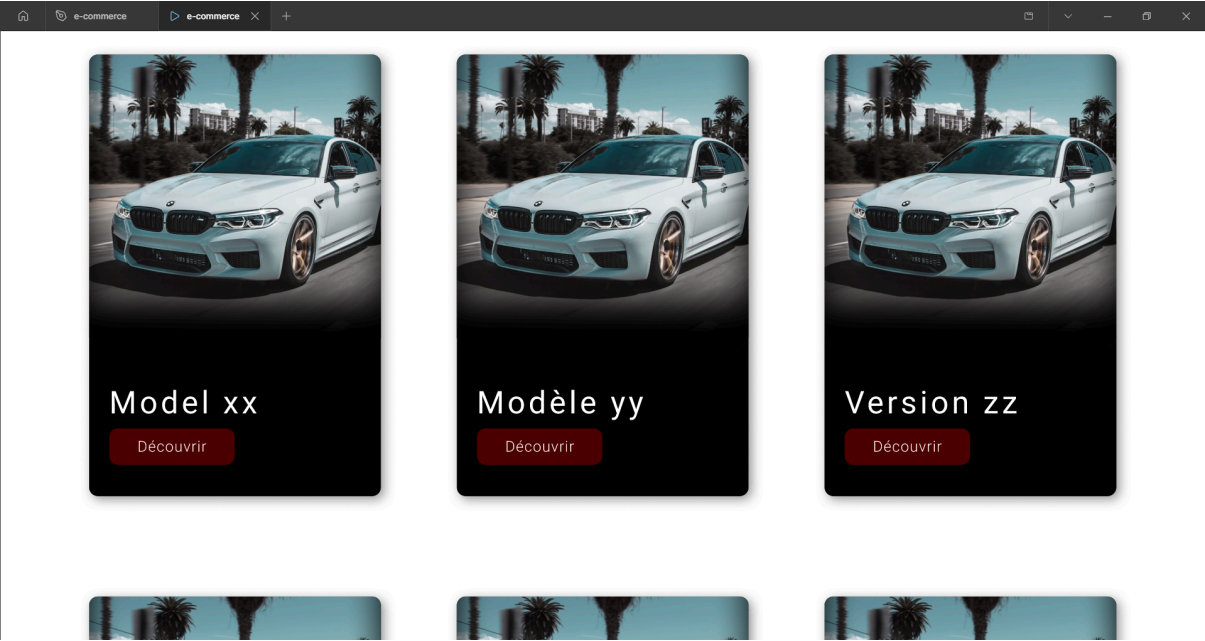
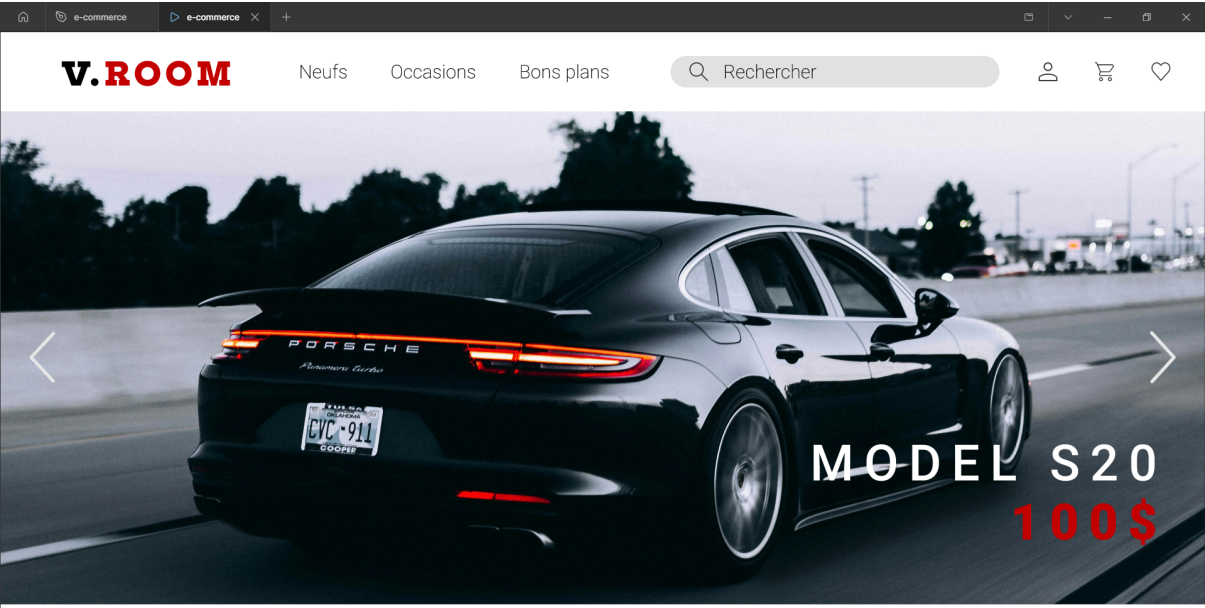
1. Identité Visuelle et Charte Graphique

L'interface de V.ROOM adopte un design moderne, épuré, conçu pour mettre en valeur les véhicules.

- **Couleurs :** Le site joue sur un contraste fort (dominance de noir, blanc et nuances de gris) avec des touches de couleur d'accentuation (comme le rouge sombre/bordeaux) pour attirer l'œil sur les éléments cliquables importants (boutons, prix).
- **Typographie :** Utilisation d'une police grasse et imposante pour le logo et les titres principaux afin d'évoquer la robustesse et le monde automobile, contrastée par une police sans empattement (sans-serif) plus classique et lisible pour les descriptions techniques.

2. Maquettes et Interface





IV. Architecture Technique

1. Modèle d'Architecture

Le projet V.ROOM repose sur une architecture découplée de type Client-Serveur communiquant via une API REST. Ce choix permet une séparation stricte des responsabilités en séparant le front-end du back-end.

2. Technologies choisies et justification

- **Frontend (Interface Utilisateur) : React.** Choisi pour sa souplesse, son approche orientée composants et la facilité de gestion d'état global avec Zustand. React s'occupe exclusivement de l'affichage et des interactions utilisateur.
- **Backend (Logique métier & API) : Symfony 7.** Framework robuste agissant comme le point d'entrée central des données. Il expose des routes d'API sécurisées que le Frontend vient interroger.
- **Base de données (Persistance) : MySQL avec l'ORM Doctrine.** MySQL assure un stockage relationnel fiable. L'utilisation de Doctrine permet de manipuler la base de données via la Programmation Orientée Objet, facilitant notamment la gestion de l'héritage des classes (Véhicules neufs / Occasions).

3. Le flux de l'Intelligence Artificielle

L'orchestration de l'IA est gérée côté serveur (Symfony) pour des raisons de sécurité et de performance.

- Le Frontend (React) envoie la requête en langage naturel de l'utilisateur.
- Symfony réceptionne la requête et construit un "Prompt" optimisé.
- Symfony interroge l'API externe (ex: OpenAI).
- L'IA retourne les critères techniques extraits (ex: type="SUV", places=5).
- Symfony traduit ces critères en requête Doctrine (SQL) pour chercher dans la base MySQL.
- Les véhicules trouvés sont renvoyés en JSON à React pour l'affichage.

V. Modèle de Données et Architecture Objet

1. Modélisation Orientée Objet (Diagramme de Classes UML)

Pour gérer le catalogue de voitures de V.ROOM, l'architecture du code repose sur les principes de la Programmation Orientée Objet (POO), et plus particulièrement sur l'héritage. Ce choix technique permet de respecter le principe DRY (*Don't Repeat Yourself*) et d'optimiser la base de données via Doctrine.

- **La Classe Mère Abstraite (**Vehicle**)** : Nous avons défini une classe abstraite **Vehicle**. Elle a pour rôle de centraliser tous les attributs communs à n'importe quelle voiture du catalogue (marque, modèle, prix, énergie, etc.). Elle est "abstraite" car un véhicule générique n'existe pas dans la réalité de la boutique : un véhicule vendu est obligatoirement soit neuf, soit d'occasion.
- **La Spécialisation (Classes Filles)** :
 - **NewVehicle (Véhicule Neuf)** : Cette classe hérite de **Vehicle**. Elle récupère automatiquement toutes les caractéristiques communes et y ajoute ses propres attributs spécifiques, comme la **dureeGarantie**.
 - **UsedVehicle (Véhicule d'Occasion)** : De la même manière, cette classe hérite de **Vehicle** mais implémente des attributs exclusifs au quickshellmarché de l'occasion, tels que le **kilometrage** et l'**etatGeneral**.

